TECNOLÓGICO DE ESTUDIOS SUPERIORES DE ECATEPEC

DIVISIÓN DE INGENIERÍA EN SISTEMAS COMPUTACIONALES

PROGRAMACIÓN DE BASE DE DATOS

TAREA 7: PROGRAMACIÓN DEL LADO DEL SERVIDOR: CREACIÓN DE VISTAS

PROFESORA:

ALEJANDRA GONZALEZ SÁNCHEZ

NOMBRE:

MONTERRUBIO LÓPEZ LUIS GAEL

GRUPO: 5701

FECHA: 30-10-26

1. Introducción y objetivo

Este informe explica qué son las vistas en una base de datos relacional, para qué sirven, sus tipos, ventajas y desventajas. Después describe cómo se diseñaron e implementaron seis vistas en PostgreSQL para el proyecto IoT Analytics Dashboard de NexoDev Solutions.

Las vistas tienen dos propósitos en el proyecto. Primero, **simplificar las consultas**: el tablero de Grafana ya no escribe uniones de cuatro tablas ni agrupaciones, solo hace `SELECT` a una vista. Segundo, **mejorar la seguridad**: cada usuario de la base de datos solo tiene permisos sobre las vistas que necesita y nunca toca las tablas. Las lecturas llegan en tiempo real desde un simulador de sensores a través de RabbitMQ.

Repositorio del proyecto: https://github.com/luisgael-ml/iot-vistas

Competencia específica: manejar un lenguaje de programación visual y la conexión a la base de datos mediante código SQL embebido (clave de actividad BDM-1301T03CE01A01).

Objetivos específicos:

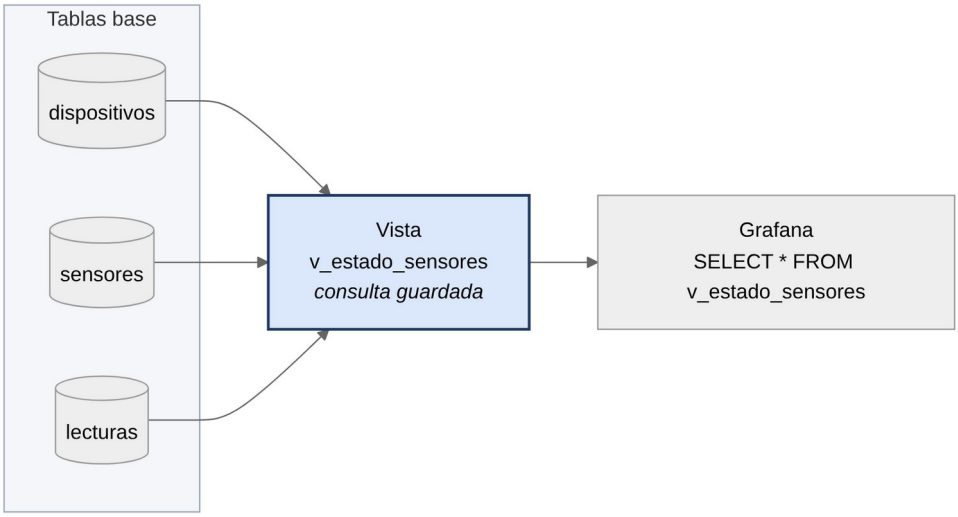
1. Analizar los requisitos de la vista en el contexto del proyecto IoT Analytics Dashboard.
2. Diseñar la estructura de las vistas, identificando las tablas y relaciones necesarias.
3. Implementar las vistas en PostgreSQL con buenas prácticas de programación SQL.
4. Configurar un entorno de desarrollo con Docker y verificar la funcionalidad de las vistas.
5. Documentar el diseño, la implementación y las pruebas realizadas.

2. Marco teórico: vistas en bases de datos

2.1 ¿Qué es una vista?

Una vista es una **consulta SQL guardada con nombre** dentro de la base de datos. Se usa como si fuera una tabla, pero no almacena datos: cada vez que alguien la consulta, el gestor ejecuta la consulta que la define sobre las **tablas base** y devuelve el resultado actualizado. Por eso también se le llama tabla virtual.

Se crea con `CREATE VIEW nombre AS SELECT ...`. A partir de ese momento cualquier usuario con permiso puede escribir `SELECT * FROM nombre`, sin conocer las tablas, las uniones ni los cálculos que hay detrás.



Una vista combina varias tablas base y las presenta como una sola tabla.

2.2 Propósito de las vistas

PROPÓSITO	DESCRIPCIÓN	EJEMPLO EN ESTE PROYECTO
Simplificar consultas	Encapsula uniones, agrupaciones y cálculos complejos detrás de un nombre	`v_salud_dispositivos` une 3 tablas, cuenta lecturas y alertas y calcula la salud
Seguridad	Se da permiso sobre la vista y no sobre las tablas; se ocultan columnas y filas	Grafana solo puede leer vistas; no ve la tabla `lecturas`
Independencia lógica	Si cambian las tablas, se ajusta la vista y las aplicaciones no se modifican	Grafana sigue funcionando aunque se agreguen columnas a `sensores`
Reglas de negocio uniformes	Un cálculo se define una sola vez y todos lo usan igual	La regla “crítica si supera el umbral en más de 20 %” vive en la vista
Rendimiento	Una vista materializada guarda el resultado de una consulta costosa	`mv_resumen_hora` resume 3 días de lecturas por hora

2.3 Tipos de vistas

TIPO	CARACTERÍSTICAS	USO EN EL PROYECTO
Simple	Se basa en una sola tabla, sin agrupaciones. PostgreSQL la vuelve actualizable automáticamente	`v_operacion_dispositivos`
Compleja	Une varias tablas, usa funciones de agregado, `GROUP BY`, subconsultas o `CASE`. Es de solo lectura	`v_estado_sensores`, `v_tendencia_24h`, `v_alertas_pendientes`, `v_salud_dispositivos`
Materializada	Guarda físicamente el resultado. Es muy rápida de leer, pero debe refrescarse con `REFRESH MATERIALIZED VIEW`	`mv_resumen_hora`
Actualizable con CHECK OPTION	Permite `INSERT`/`UPDATE` a través de la vista, pero rechaza filas que quedarían fuera de su filtro `WHERE`	`v_operacion_dispositivos`
Con barrera de seguridad	`security_barrier` evita que funciones del usuario se evalúen antes que los filtros de la vista y filtren datos ocultos	`v_estado_sensores`

2.4 Ventajas y desventajas

VENTAJAS	DESVENTAJAS
Consultas más cortas y fáciles de leer para quien las usa	Una vista no es más rápida que su consulta: si la consulta es lenta, la vista también
Control de acceso fino: columnas y filas visibles por usuario	Vistas anidadas sobre otras vistas son difíciles de optimizar y depurar
Reglas y cálculos definidos en un solo lugar	Las vistas complejas no se pueden actualizar directamente
Las aplicaciones no dependen de la estructura de las tablas	Una vista materializada muestra datos con retraso hasta que se refresca
Las vistas materializadas aceleran reportes históricos	Cambiar una tabla puede romper las vistas que dependen de ella

2.5 Vista, vista materializada y tabla

ASPECTO	VISTA	VISTA MATERIALIZADA	TABLA
Guarda datos	No	Sí (resultado de la consulta)	Sí
Datos actualizados	Siempre	Hasta el último REFRESH	Siempre
Velocidad de lectura	Igual a su consulta	Muy alta	Muy alta
Admite índices	No (usa los de las tablas)	Sí	Sí
Uso típico	Simplificar y proteger	Reportes e históricos	Almacenar datos

2.6 Ejemplos en sistemas reales

SISTEMA	VISTA	PARA QUÉ
Banca	Movimientos del cliente sin datos internos del banco	El cajero ve saldo y movimientos, no comisiones internas
Recursos humanos	Empleados sin la columna de salario	Jefes de área consultan a su equipo sin ver sueldos
Comercio electrónico	Ventas diarias por producto (materializada)	Reportes rápidos sin recorrer millones de pedidos
ERP (SAP)	CDS Views sobre las tablas de finanzas	Reportes y aplicaciones Fiori sin acceso directo a las tablas
IoT Analytics Dashboard	Estado, tendencias, alertas y salud de dispositivos	Grafana solo lee vistas; los operadores solo cambian estados permitidos

3. Análisis de requisitos

Antes de crear las vistas se identificaron las preguntas que el tablero de NexoDev debe responder y las restricciones de seguridad de cada usuario del sistema.

NO.	REQUISITO	TIPO	VISTA QUE LO CUMPLE
R1	Ver la última lectura de cada sensor y si está normal, en advertencia o crítica	Consulta	`v_estado_sensores`
R2	Graficar la tendencia por minuto de cada sensor en las últimas 24 horas	Consulta	`v_tendencia_24h`
R3	Listar las alertas sin atender, primero las críticas, con su porcentaje de exceso	Consulta	`v_alertas_pendientes`

NO.	REQUISITO	TIPO	VISTA QUE LO CUMPLE
R4	Resumir la salud de cada dispositivo: lecturas, alertas y estado	Consulta	`v_salud_dispositivos`
R5	Consultar el histórico de días sin que el tablero se vuelva lento	Rendimiento	`mv_resumen_hora`
R6	Grafana debe tener solo lectura y no acceder a las tablas	Seguridad	Permisos solo sobre vistas
R7	El operador solo puede poner un equipo en activo o mantenimiento, nunca darlo de baja	Seguridad	`v_operacion_dispositivos` con CHECK OPTION
R8	El consumidor de RabbitMQ solo inserta lecturas y refresca el resumen	Seguridad	Permiso `INSERT` y función `SECURITY DEFINER`

4. Diseño

4.1 Tablas y relaciones

Las vistas se construyen sobre las cuatro tablas del proyecto. Un dispositivo tiene varios sensores; cada sensor registra muchas lecturas; y una lectura que supera el umbral del sensor origina una alerta. Un trigger crea la alerta automáticamente al insertar la lectura.

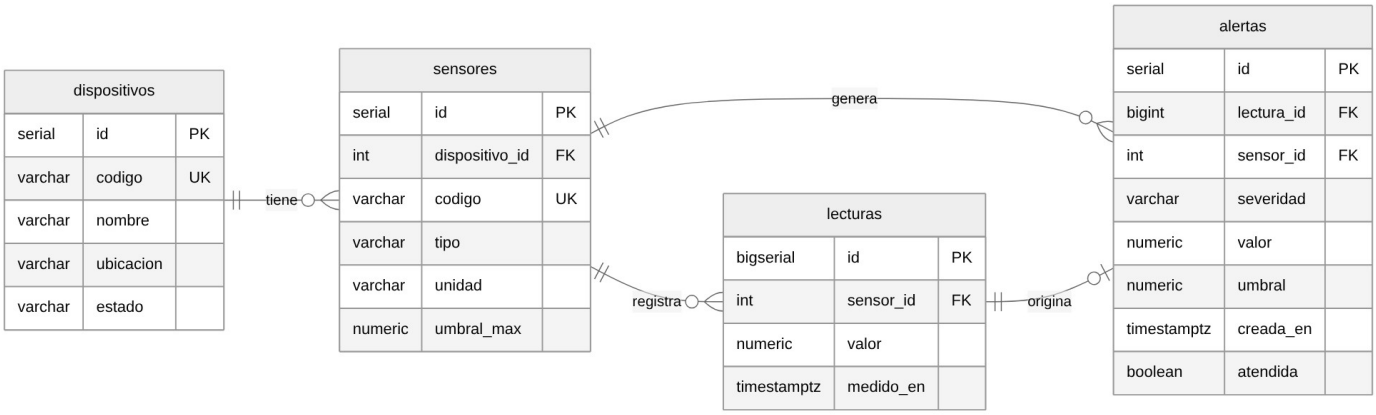


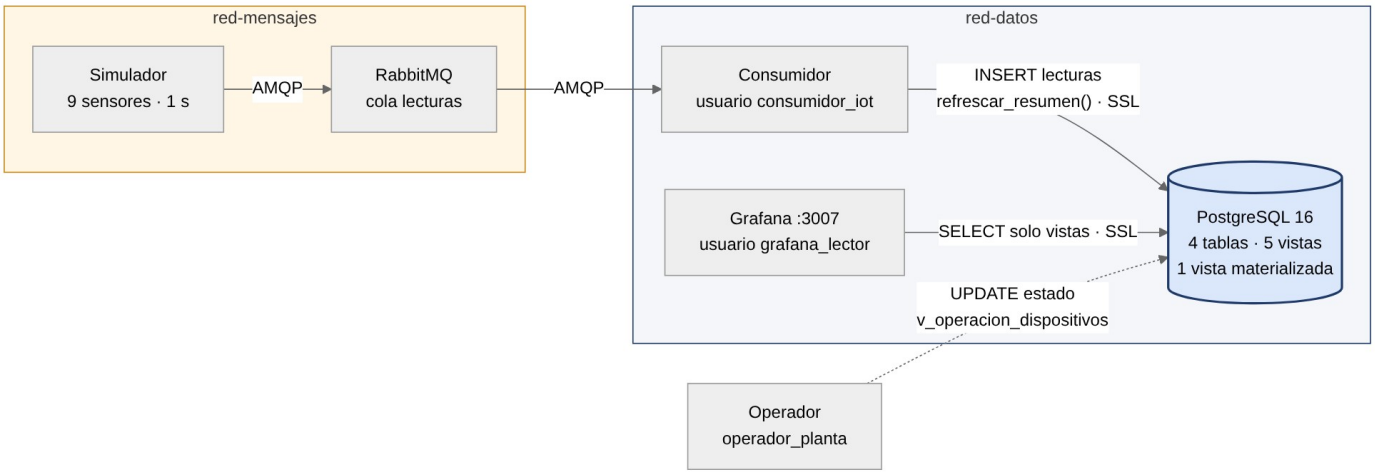
Diagrama entidad-relación de las tablas base.

4.2 Diseño de las vistas

VISTA	TABLAS	TÉCNICA PRINCIPAL	DECISIÓN DE DISEÑO
`v_estado_sensores`	sensores, dispositivos, lecturas	`LEFT JOIN LATERAL ... LIMIT 1`	Lee una sola fila por sensor usando el índice (sensor_id, medido_en DESC), en vez de agrupar todas las lecturas
`v_tendencia_24h`	lecturas, sensores, dispositivos	`date_trunc` + `GROUP BY`	El filtro de 24 h va dentro de la vista para usar el índice por fecha
`v_alertas_pendientes`	alertas, sensores, dispositivos	Columnas calculadas, `ORDER BY`	Calcula el % de exceso y los minutos abierta; ordena críticas primero
`v_salud_dispositivos`	dispositivos, sensores, alertas, lecturas	`COUNT ... FILTER`, subconsulta, `CASE`	Una fila por dispositivo con su salud: ok, vigilar o en riesgo
`mv_resumen_hora`	lecturas, sensores	Vista materializada + índice único	Permite `REFRESH CONCURRENTLY`: Grafana lee mientras se actualiza
`v_operacion_dispositivos`	dispositivos	Vista simple `WITH CASCADED CHECK OPTION`	Solo muestra equipos en operación y bloquea el estado inactivo

4.3 Arquitectura y entorno

El sistema corre en cinco contenedores de Docker separados en dos redes. El simulador y RabbitMQ están en red-mensajes; PostgreSQL y Grafana están en red-datos. El consumidor es el único en ambas redes: toma cada lectura de la cola y la inserta en la base de datos. La base de datos no expone puertos a la PC y todas las conexiones usan SSL.

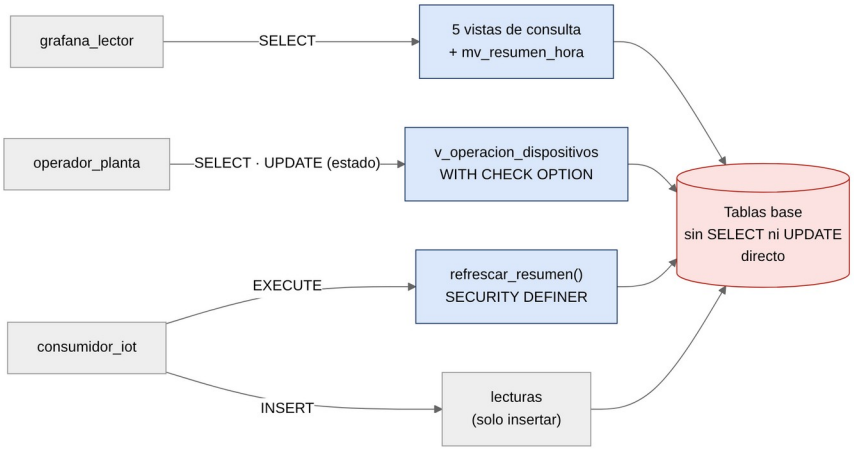


Arquitectura del sistema con sus redes aisladas.

CONTENEDOR	IMAGEN	RED	PUERTO EN LA PC
vistas-datos	postgres:16 (SSL)	red-datos	Ninguno
vistas-rabbitmq	rabbitmq:3.13-management	red-mensajes	15677 (consola)
vistas-consumidor	iot-vistas-servicios:1.0 (Python 3.12)	red-datos y red-mensajes	Ninguno
vistas-simulador	iot-vistas-servicios:1.0 (Python 3.12)	red-mensajes	Ninguno
vistas-grafana	grafana/grafana:11.3.0	red-datos	3007

4.4 Seguridad con vistas y permisos

Se crearon tres usuarios con el mínimo privilegio. Ninguno tiene `SELECT` ni `UPDATE` directo sobre las tablas: todo su acceso pasa por vistas o por una función controlada.



Permisos de cada usuario: el acceso a las tablas siempre pasa por una vista o función.

USUARIO	PUEDE	NO PUEDE
`grafana_lector`	Leer las 5 vistas de consulta y `mv_resumen_hora`	Leer o modificar cualquier tabla
`operador_planta`	Ver equipos en operación y cambiar su estado a activo o mantenimiento	Dar de baja un equipo, cambiar otras columnas o tocar la tabla `dispositivos`
`consumidor_iot`	Insertar lecturas y ejecutar `refrescar_resumen()`	Leer lecturas, modificar o borrar datos

5. Implementación

5.1 Estructura del proyecto

```
iot-vistas/
├── docker-compose.yml        → base de datos, RabbitMQ, consumidor, simulador y Grafana
├── datos/
│   ├── init.sql              → tablas, trigger, 6 vistas, usuarios, permisos y datos
│   └── pruebas.sql           → pruebas de rendimiento con EXPLAIN ANALYZE
├── servicios/
│   ├── Dockerfile, requirements.txt
│   ├── consumidor.py         → RabbitMQ → PostgreSQL y refresco de la vista materializada
│   └── simulador.py          → 9 sensores IoT que publican en RabbitMQ
└── grafana/
    ├── provisioning/         → origen de datos (grafana_lector, SSL) y tablero
    └── dashboards/iot-vistas.json → tablero de 11 paneles, todos sobre vistas
```

5.2 Base de datos y vistas: init.sql

Buenas prácticas aplicadas: nombres con prefijo (`v_` para vistas, `mv_` para materializadas), alias de columnas claros, índices que apoyan los filtros de las vistas, funciones `SECURITY DEFINER` con `search_path` fijo y comentarios que justifican cada decisión.

```
-- ================================================================
-- TAREA 7 · Programacion del lado del servidor: creacion de vistas
-- IoT Analytics Dashboard (NexoDev Solutions) · PostgreSQL 16
-- Las vistas simplifican las consultas de Grafana y limitan lo que cada
-- usuario puede ver o modificar. Nadie (salvo el administrador) toca las tablas.
-- ================================================================

-- ------------------------------ TABLAS ------------------------------
CREATE TABLE dispositivos (
    id        SERIAL PRIMARY KEY,
    codigo    VARCHAR(20) NOT NULL UNIQUE,
    nombre    VARCHAR(60) NOT NULL,
    ubicacion VARCHAR(60) NOT NULL,
    estado    VARCHAR(15) NOT NULL DEFAULT 'activo'
              CHECK (estado IN ('activo', 'mantenimiento', 'inactivo'))
);

CREATE TABLE sensores (
    id             SERIAL PRIMARY KEY,
    dispositivo_id INT NOT NULL REFERENCES dispositivos(id),
    codigo         VARCHAR(20) NOT NULL UNIQUE,
    tipo           VARCHAR(20) NOT NULL CHECK (tipo IN ('temperatura', 'vibracion', 'energia')),
    unidad         VARCHAR(10) NOT NULL,
    umbral_max     NUMERIC(10,2) NOT NULL
);

CREATE TABLE lecturas (
    id        BIGSERIAL PRIMARY KEY,
    sensor_id INT NOT NULL REFERENCES sensores(id),
```

```
    valor     NUMERIC(10,2) NOT NULL,
    medido_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- Indices que aprovechan las vistas: ultima lectura por sensor y filtros por fecha
CREATE INDEX idx_lecturas_sensor_fecha ON lecturas (sensor_id, medido_en DESC);
CREATE INDEX idx_lecturas_fecha        ON lecturas (medido_en DESC);

CREATE TABLE alertas (
    id         SERIAL PRIMARY KEY,
    lectura_id BIGINT NOT NULL REFERENCES lecturas(id),
    sensor_id  INT NOT NULL REFERENCES sensores(id),
    severidad  VARCHAR(12) NOT NULL CHECK (severidad IN ('advertencia', 'critica')),
    valor      NUMERIC(10,2) NOT NULL,
    umbral     NUMERIC(10,2) NOT NULL,
    creada_en  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    atendida   BOOLEAN NOT NULL DEFAULT FALSE
);
CREATE INDEX idx_alertas_pendientes ON alertas (creada_en DESC) WHERE NOT atendida;

-- Alerta automatica: advertencia si supera el umbral, critica si lo supera en mas de 20 %
CREATE FUNCTION generar_alerta() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
    v_umbral NUMERIC;
BEGIN
    SELECT umbral_max INTO v_umbral FROM sensores WHERE id = NEW.sensor_id;
    IF NEW.valor > v_umbral THEN
        INSERT INTO alertas (lectura_id, sensor_id, severidad, valor, umbral, creada_en)
        VALUES (NEW.id, NEW.sensor_id,
                CASE WHEN NEW.valor > v_umbral * 1.2 THEN 'critica' ELSE 'advertencia' END,
                NEW.valor, v_umbral, NEW.medido_en);
    END IF;
    RETURN NEW;
END $$;

CREATE TRIGGER trg_generar_alerta AFTER INSERT ON lecturas
FOR EACH ROW EXECUTE FUNCTION generar_alerta();

-- ------------------------------ VISTAS ------------------------------

-- V1. Ultima lectura de cada sensor y su situacion.
-- LATERAL + LIMIT 1 usa el indice (sensor_id, medido_en DESC): lee 1 fila por sensor.
-- security_barrier: los filtros del usuario no se evaluan antes que los de la vista.
CREATE VIEW v_estado_sensores WITH (security_barrier) AS
SELECT s.codigo     AS sensor,
       s.tipo,
       s.unidad,
       d.codigo     AS dispositivo,
       d.ubicacion,
       d.estado     AS estado_dispositivo,
       s.umbral_max AS umbral,
       u.valor,
       u.medido_en,
       ROUND(100 * u.valor / s.umbral_max, 1) AS pct_umbral,
       CASE WHEN u.valor IS NULL                THEN 'sin datos'
            WHEN u.valor > s.umbral_max * 1.2  THEN 'critica'
            WHEN u.valor > s.umbral_max        THEN 'advertencia'
            ELSE 'normal' END                AS situacion
FROM sensores s
JOIN dispositivos d ON d.id = s.dispositivo_id
LEFT JOIN LATERAL (
    SELECT l.valor, l.medido_en
    FROM lecturas l
    WHERE l.sensor_id = s.id
    ORDER BY l.medido_en DESC
    LIMIT 1
) u ON TRUE;
```

```
-- V2. Tendencia por minuto de las ultimas 24 h (graficas de Grafana).
-- El filtro de 24 h va DENTRO de la vista para que use idx_lecturas_fecha.
CREATE VIEW v_tendencia_24h AS
SELECT date_trunc('minute', l.medido_en) AS minuto,
       s.codigo  AS sensor,
       s.tipo,
       d.codigo  AS dispositivo,
       COUNT(*)              AS lecturas,
       ROUND(AVG(l.valor), 2) AS promedio,
       MIN(l.valor)          AS minimo,
       MAX(l.valor)          AS maximo
FROM lecturas l
JOIN sensores s     ON s.id = l.sensor_id
JOIN dispositivos d ON d.id = s.dispositivo_id
WHERE l.medido_en > NOW() - INTERVAL '24 hours'
GROUP BY 1, 2, 3, 4;

-- V3. Alertas pendientes con contexto, ordenadas por prioridad.
CREATE VIEW v_alertas_pendientes AS
SELECT a.id,
       a.creada_en,
       s.codigo    AS sensor,
       s.tipo,
       d.codigo    AS dispositivo,
       d.ubicacion,
       a.severidad,
       a.valor,
       a.umbral,
       ROUND(100 * (a.valor - a.umbral) / a.umbral, 1)              AS pct_exceso,
       ROUND(EXTRACT(EPOCH FROM NOW() - a.creada_en) / 60, 1)       AS minutos_abierta
FROM alertas a
JOIN sensores s     ON s.id = a.sensor_id
JOIN dispositivos d ON d.id = s.dispositivo_id
WHERE NOT a.atendida
ORDER BY (a.severidad = 'critica') DESC, a.creada_en DESC;

-- V4. Salud de cada dispositivo (resumen para el tablero).
CREATE VIEW v_salud_dispositivos AS
SELECT d.codigo AS dispositivo,
       d.nombre,
       d.ubicacion,
       d.estado,
       COUNT(DISTINCT s.id)                                            AS sensores,
       (SELECT COUNT(*) FROM lecturas l JOIN sensores s2 ON s2.id = l.sensor_id
         WHERE s2.dispositivo_id = d.id
           AND l.medido_en > NOW() - INTERVAL '1 hour')                AS lecturas_ultima_hora,
       COUNT(a.id)                                                     AS alertas_pendientes,
       COUNT(a.id) FILTER (WHERE a.severidad = 'critica')              AS alertas_criticas,
       CASE WHEN d.estado <> 'activo'                                  THEN d.estado
            WHEN COUNT(a.id) FILTER (WHERE a.severidad = 'critica') > 0 THEN 'en riesgo'
            WHEN COUNT(a.id) > 0                                       THEN 'vigilar'
            ELSE 'ok' END                                              AS salud
FROM dispositivos d
JOIN sensores s     ON s.dispositivo_id = d.id
LEFT JOIN alertas a ON a.sensor_id = s.id AND NOT a.atendida
GROUP BY d.id, d.codigo, d.nombre, d.ubicacion, d.estado;

-- V5. Vista materializada: estadisticas por hora (historico).
-- Guarda el resultado; consultar 3 dias de historia no recorre toda la tabla de lecturas.
CREATE MATERIALIZED VIEW mv_resumen_hora AS
SELECT date_trunc('hour', l.medido_en) AS hora,
       s.codigo  AS sensor,
       s.tipo,
       COUNT(*)                AS lecturas,
       ROUND(AVG(l.valor), 2)   AS promedio,
       MIN(l.valor)             AS minimo,
       MAX(l.valor)             AS maximo,
       ROUND(percentile_cont(0.95) WITHIN GROUP (ORDER BY l.valor)::numeric, 2) AS p95,
```

```
       COUNT(*) FILTER (WHERE l.valor > s.umbral_max) AS lecturas_sobre_umbral
FROM lecturas l
JOIN sensores s ON s.id = l.sensor_id
GROUP BY 1, 2, 3
WITH NO DATA;
-- Indice unico: permite REFRESH ... CONCURRENTLY (Grafana puede leer mientras se actualiza)
CREATE UNIQUE INDEX ux_resumen_hora ON mv_resumen_hora (hora, sensor);

-- El consumidor refresca la vista sin ser su dueno: funcion SECURITY DEFINER
CREATE FUNCTION refrescar_resumen() RETURNS void
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
BEGIN
    REFRESH MATERIALIZED VIEW CONCURRENTLY mv_resumen_hora;
END $$;

-- V6. Vista actualizable para operadores de planta.
-- Solo expone dispositivos en operacion y solo deja cambiar el estado;
-- WITH CHECK OPTION impide dar de baja (inactivo) desde la vista.
CREATE VIEW v_operacion_dispositivos AS
SELECT codigo, nombre, ubicacion, estado
FROM dispositivos
WHERE estado IN ('activo', 'mantenimiento')
WITH CASCADED CHECK OPTION;

-- ------------------------ USUARIOS Y PERMISOS ------------------------
-- Grafana: solo lectura y SOLO sobre las vistas (no ve las tablas)
CREATE ROLE grafana_lector LOGIN PASSWORD 'grafana123';
GRANT SELECT ON v_estado_sensores, v_tendencia_24h, v_alertas_pendientes,
                v_salud_dispositivos, mv_resumen_hora TO grafana_lector;

-- Consumidor de RabbitMQ: inserta lecturas y refresca el resumen
CREATE ROLE consumidor_iot LOGIN PASSWORD 'consumidor123';
GRANT SELECT (id, codigo) ON sensores TO consumidor_iot;
GRANT INSERT ON lecturas TO consumidor_iot;
GRANT USAGE ON SEQUENCE lecturas_id_seq TO consumidor_iot;
REVOKE EXECUTE ON FUNCTION refrescar_resumen() FROM PUBLIC;
GRANT EXECUTE ON FUNCTION refrescar_resumen() TO consumidor_iot;

-- Operador de planta: consulta y cambia el estado solo a traves de V6
CREATE ROLE operador_planta LOGIN PASSWORD 'operador123';
GRANT SELECT, UPDATE (estado) ON v_operacion_dispositivos TO operador_planta;

-- -------------------------- DATOS INICIALES --------------------------
INSERT INTO dispositivos (codigo, nombre, ubicacion) VALUES
('PRE-03', 'Prensa hidraulica 3', 'Linea 1'),
('TOR-01', 'Torno CNC 1',         'Linea 2'),
('EMB-02', 'Embotelladora 2',     'Linea 3');

INSERT INTO sensores (dispositivo_id, codigo, tipo, unidad, umbral_max) VALUES
(1, 'PRE03-TEMP', 'temperatura', '°C',   85.00),
(1, 'PRE03-VIB',  'vibracion',   'mm/s',  7.00),
(1, 'PRE03-ENE',  'energia',     'kWh',  30.00),
(2, 'TOR01-TEMP', 'temperatura', '°C',   80.00),
(2, 'TOR01-VIB',  'vibracion',   'mm/s',  6.00),
(2, 'TOR01-ENE',  'energia',     'kWh',  25.00),
(3, 'EMB02-TEMP', 'temperatura', '°C',   75.00),
(3, 'EMB02-VIB',  'vibracion',   'mm/s',  5.00),
(3, 'EMB02-ENE',  'energia',     'kWh',  20.00);

-- Historico de 3 dias (una lectura cada 30 s por sensor) para probar el rendimiento
SELECT setseed(0.7);
INSERT INTO lecturas (sensor_id, valor, medido_en)
SELECT s.id,
       ROUND((CASE WHEN random() < 0.003 THEN s.umbral_max * (1.05 + random() * 0.3)
                   ELSE s.umbral_max * (0.80 + (random() - 0.5) * 0.12) END)::numeric, 2),
       t
FROM sensores s
CROSS JOIN generate_series(NOW() - INTERVAL '3 days', NOW() - INTERVAL '1 minute',
```

```
                            INTERVAL '30 seconds') AS t;
UPDATE alertas SET atendida = TRUE;   -- las alertas historicas ya fueron atendidas

REFRESH MATERIALIZED VIEW mv_resumen_hora;
ANALYZE;
```

5.3 Consumidor de RabbitMQ

Inserta cada lectura con el usuario `consumidor_iot` mediante una consulta parametrizada (evita inyección SQL) y cada 30 segundos refresca `mv_resumen_hora` llamando a la función `refrescar_resumen()`.

```
"""
Consumidor: toma las lecturas de RabbitMQ y las guarda en PostgreSQL con el
usuario consumidor_iot (solo puede insertar lecturas). Cada 30 s refresca la
vista materializada mv_resumen_hora mediante la funcion refrescar_resumen().
"""
import json
import os
import time

import pika
import psycopg

CONNINFO = (f"host={os.getenv('DB_HOST', 'datos')} dbname=iotanalytics "
            f"user={os.getenv('DB_USER', 'consumidor_iot')} "
            f"password={os.environ['DB_PASSWORD']} sslmode=require")
REFRESCO = int(os.getenv("REFRESCO_SEG", "30"))

INSERTAR = """INSERT INTO lecturas (sensor_id, valor, medido_en)
              SELECT id, %s, %s FROM sensores WHERE codigo = %s"""


def conectar_bd():
    while True:
        try:
            return psycopg.connect(CONNINFO, autocommit=True)
        except psycopg.OperationalError:
            print("PostgreSQL no disponible, reintentando...")
            time.sleep(3)


def conectar_rabbit():
    params = pika.URLParameters(os.getenv("RABBIT_URL", "amqp://iot:iot123@mensajes:5672/"))
    while True:
        try:
            return pika.BlockingConnection(params)
        except pika.exceptions.AMQPConnectionError:
            print("RabbitMQ no disponible, reintentando...")
            time.sleep(3)


def main():
    bd = conectar_bd()
    print("Conectado a PostgreSQL con SSL:", bd.info.pgconn.ssl_in_use)
    con = conectar_rabbit()
    canal = con.channel()
    canal.queue_declare(queue="lecturas", durable=True)
    canal.basic_qos(prefetch_count=50)
    guardadas = 0

    def al_recibir(ch, metodo, _props, cuerpo):
        nonlocal guardadas
        lectura = json.loads(cuerpo)
        filas = bd.execute(INSERTAR, (lectura["valor"], lectura["medido_en"],
```

```
                                        lectura["sensor"])).rowcount
        if filas == 0:
            print(f"Sensor desconocido: {lectura['sensor']}")
        guardadas += filas
        ch.basic_ack(delivery_tag=metodo.delivery_tag)

    def refrescar():
        inicio = time.perf_counter()
        bd.execute("SELECT refrescar_resumen()")
        print(f"{guardadas} lecturas guardadas · mv_resumen_hora refrescada en "
              f"{(time.perf_counter() - inicio) * 1000:.0f} ms")
        con.call_later(REFRESCO, refrescar)

    con.call_later(REFRESCO, refrescar)
    canal.basic_consume(queue="lecturas", on_message_callback=al_recibir)
    print("Consumidor escuchando la cola 'lecturas'")
    canal.start_consuming()


if __name__ == "__main__":
    main()
```

5.4 Simulador de sensores

```
"""
Simulador de sensores IoT: publica una lectura por sensor cada segundo
en la cola 'lecturas' de RabbitMQ. Nunca se conecta a la base de datos.
"""
import json
import os
import random
import time
from datetime import datetime, timezone

import pika

INTERVALO = float(os.getenv("INTERVALO", "1"))
# sensor -> (valor base, variacion, umbral)
SENSORES = {
    "PRE03-TEMP": (70, 3, 85),  "PRE03-VIB": (4.5, 0.6, 7), "PRE03-ENE": (22, 2, 30),
    "TOR01-TEMP": (65, 3, 80),  "TOR01-VIB": (3.8, 0.5, 6), "TOR01-ENE": (18, 2, 25),
    "EMB02-TEMP": (60, 3, 75),  "EMB02-VIB": (3.0, 0.4, 5), "EMB02-ENE": (14, 2, 20),
}


def conectar():
    params = pika.URLParameters(os.getenv("RABBIT_URL", "amqp://iot:iot123@mensajes:5672/"))
    while True:
        try:
            return pika.BlockingConnection(params)
        except pika.exceptions.AMQPConnectionError:
            print("RabbitMQ no disponible, reintentando...")
            time.sleep(3)


def main():
    con = conectar()
    canal = con.channel()
    canal.queue_declare(queue="lecturas", durable=True)
    print(f"Simulador publicando en la cola 'lecturas' cada {INTERVALO}s")
    enviados = 0
    while True:
        for sensor, (base, variacion, umbral) in SENSORES.items():
            valor = random.gauss(base, variacion)
            r = random.random()
            if r < 0.02:
                valor = umbral * random.uniform(1.02, 1.15)   # pico: advertencia
```

```
            elif r < 0.03:
                valor = umbral * random.uniform(1.25, 1.40)   # pico: critica
            mensaje = {"sensor": sensor, "valor": round(valor, 2),
                       "medido_en": datetime.now(timezone.utc).isoformat()}
            canal.basic_publish(exchange="", routing_key="lecturas", body=json.dumps(mensaje),
                                properties=pika.BasicProperties(delivery_mode=2))
            enviados += 1
        if enviados % 900 == 0:
            print(f"{enviados} lecturas publicadas")
        time.sleep(INTERVALO)


if __name__ == "__main__":
    main()
```

5.5 Orquestación con Docker Compose

```
# Tarea 7 · Creacion de vistas · IoT Analytics Dashboard (NexoDev Solutions)
name: iot-vistas

services:
  # ================= BASE DE DATOS (tablas + vistas) =================
  datos:
    image: postgres:16
    container_name: vistas-datos
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD:-admin123}
      POSTGRES_DB: iotanalytics
    command: >
      -c ssl=on -c ssl_cert_file=/etc/ssl/certs/ssl-cert-snakeoil.pem
      -c ssl_key_file=/etc/ssl/private/ssl-cert-snakeoil.key
    volumes:
      - ./datos/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    networks: [red-datos]          # sin puerto expuesto a la PC
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d iotanalytics"]
      interval: 3s
      retries: 30

  # ================= MENSAJERIA EN TIEMPO REAL =================
  mensajes:
    image: rabbitmq:3.13-management
    container_name: vistas-rabbitmq
    hostname: vistas-rabbitmq      # nombre fijo del nodo
    environment:
      RABBITMQ_DEFAULT_USER: iot
      RABBITMQ_DEFAULT_PASS: iot123
      # La cookie de Erlang va como argumento: RabbitMQ ya no lee
      # /var/lib/rabbitmq/.erlang.cookie, que falla con "eacces" en Docker Desktop
      RABBITMQ_SERVER_ADDITIONAL_ERL_ARGS: "-setcookie iotvistascookie"
    tmpfs:
      - /var/lib/rabbitmq:mode=1777
    ports:
      - "15677:15672"              # consola de RabbitMQ: http://localhost:15677
    networks: [red-mensajes]
    healthcheck:
      # Prueba que el puerto AMQP acepte conexiones (no usa la cookie)
      test: ["CMD-SHELL", "bash -c '</dev/tcp/localhost/5672'"]
      interval: 5s
      retries: 30

  # ================= CONSUMIDOR: RabbitMQ -> PostgreSQL =================
  consumidor:
    build: ./servicios
    image: iot-vistas-servicios:1.0
    container_name: vistas-consumidor
    command: python -u consumidor.py
```

```
    environment:
      DB_HOST: datos
      DB_USER: consumidor_iot
      DB_PASSWORD: consumidor123
      RABBIT_URL: amqp://iot:iot123@mensajes:5672/
    networks: [red-datos, red-mensajes]
    depends_on:
      datos: { condition: service_healthy }
      mensajes: { condition: service_healthy }

  # ================= SIMULADOR DE SENSORES =================
  simulador:
    build: ./servicios
    image: iot-vistas-servicios:1.0
    container_name: vistas-simulador
    command: python -u simulador.py
    environment:
      RABBIT_URL: amqp://iot:iot123@mensajes:5672/
      INTERVALO: "1"
    networks: [red-mensajes]       # no tiene acceso a la base de datos
    depends_on:
      consumidor: { condition: service_started }

  # ================= VISUALIZACION =================
  grafana:
    image: grafana/grafana:11.3.0
    container_name: vistas-grafana
    environment:
      GF_SECURITY_ADMIN_PASSWORD: admin123
      GF_AUTH_ANONYMOUS_ENABLED: "true"
      GF_AUTH_ANONYMOUS_ORG_ROLE: Viewer
      GF_DASHBOARDS_DEFAULT_HOME_DASHBOARD_PATH: /var/lib/grafana/dashboards/iot-vistas.json
    ports:
      - "3007:3000"                # http://localhost:3007
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
      - ./grafana/dashboards:/var/lib/grafana/dashboards:ro
    networks: [red-datos]
    depends_on:
      datos: { condition: service_healthy }

networks:
  red-datos:
  red-mensajes:
```

5.6 Grafana

El origen de datos usa el usuario `grafana_lector` con SSL. Los 11 paneles del tablero consultan solo vistas; por ejemplo, la gráfica de temperatura:

```
SELECT minuto AS time, sensor AS metric, promedio AS value
FROM v_tendencia_24h
WHERE tipo = 'temperatura' AND $__timeFilter(minuto)
ORDER BY 1
```

Sin la vista, la misma gráfica necesitaría unir `lecturas`, `sensores` y `dispositivos`, truncar las fechas y agrupar en cada panel.

6. Pruebas y evidencias

```
docker compose up -d --build
```

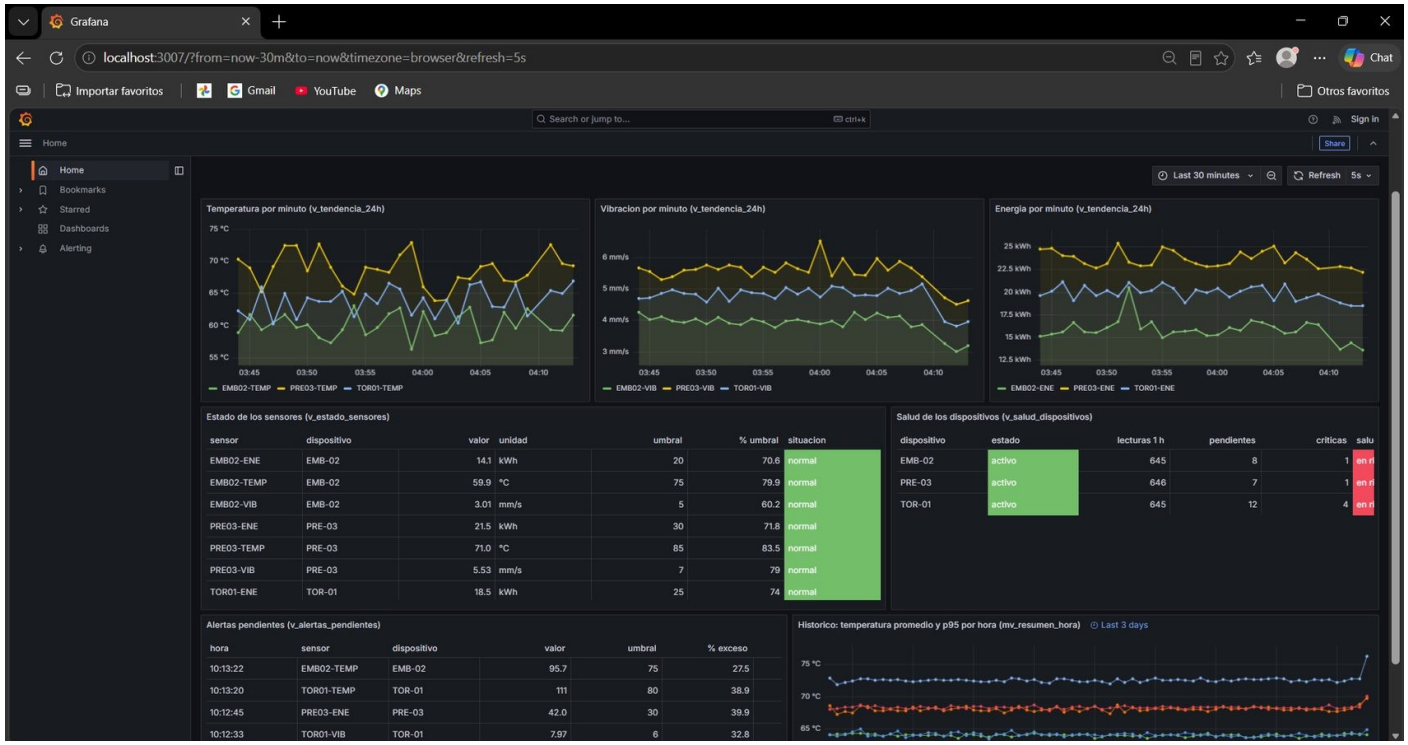
NO.	PRUEBA	RESULTADO ESPERADO	RESULTADO OBTENIDO
P1	Arranque del entorno	5 contenedores en ejecución	Correcto: datos y RabbitMQ healthy; consumidor, simulador y Grafana iniciados
P2	Tablero sobre vistas	Gráficas y tablas con datos en tiempo real	Correcto: 3 gráficas por minuto, estado de 9 sensores, salud y alertas
P3	Creación de las vistas	5 vistas y 1 vista materializada	Correcto: `\dv` y `\dm` las listan; `v_estado_sensores` devuelve 9 filas
P4	Seguridad de lectura	`grafana_lector` no lee tablas, sí vistas	Correcto: permission denied en `lecturas`; 9 filas en la vista
P5	WITH CHECK OPTION y permisos	Cambiar a mantenimiento sí; a inactivo no; tabla no	Correcto: UPDATE 1; violates check option; permission denied
P6	Cambio reflejado en el tablero	TOR-01 en mantenimiento	Correcto: TOR-01 en azul; 2 dispositivos activos, 61 alertas (19 críticas)
P7	Rendimiento de la vista materializada	Menor tiempo que la consulta directa	Correcto: 476.5 ms directa contra 2.4 ms en `mv_resumen_hora` (≈197 veces menos)
P8	Tiempo real con RabbitMQ	Mensajes consumidos y vista refrescada	Correcto: 8.8 msg/s, cola en 0; 3,123 lecturas y refresco de 360 a 1,652 ms

Evidencia 1 — P1: los cinco contenedores en ejecución.



Arranque del entorno con Docker Compose.

Evidencia 2 — P2: el tablero de Grafana grafica la tendencia por minuto y muestra el estado de los sensores, la salud de los dispositivos y las alertas, todo a partir de vistas.



Tablero IoT Analytics Dashboard construido sobre vistas.

Evidencia 3 — P3: las cinco vistas, la vista materializada y el resultado de `v_estado_sensores`.

```
C:\Users\ASLHY\Downloads\iot-vistas\iot-vistas>
C:\Users\ASLHY\Downloads\iot-vistas\iot-vistas>docker exec vistas-datos psql -U postgres -d iotanalytics -c "\dv" -c "\dm" -c "SELECT * FROM v_estado_sensor
es"
                List of relations
 Schema |           Name           | Type |  Owner
--------+--------------------------+------+----------
 public | v_alertas_pendientes     | view | postgres
 public | v_estado_sensores        | view | postgres
 public | v_operacion_dispositivos | view | postgres
 public | v_salud_dispositivos     | view | postgres
 public | v_tendencia_24h          | view | postgres
(5 rows)

                  List of relations
 Schema |      Name       |       Type        |  Owner
--------+-----------------+-------------------+----------
 public | mv_resumen_hora | materialized view | postgres
(1 row)

   sensor    |    tipo     | unidad | dispositivo | ubicacion | estado_dispositivo | umbral | valor |          medido_en           | pct_umbral | situacion
-------------+-------------+--------+-------------+-----------+--------------------+--------+-------+------------------------------+------------+-----------
 PRE03-TEMP  | temperatura | °C     | PRE-03      | Linea 1   | activo             |  85.00 | 73.97 | 2026-10-30 10:13:47.616064+00 |       87.0 | normal
 PRE03-VIB   | vibracion   | mm/s   | PRE-03      | Linea 1   | activo             |   7.00 |  4.98 | 2026-10-30 10:13:47.617029+00 |       71.1 | normal
 PRE03-ENE   | energia     | kWh    | PRE-03      | Linea 1   | activo             |  30.00 | 20.74 | 2026-10-30 10:13:46.585754+00 |       69.1 | normal
 TOR01-TEMP  | temperatura | °C     | TOR-01      | Linea 2   | activo             |  80.00 | 64.15 | 2026-10-30 10:13:46.586146+00 |       80.2 | normal
 TOR01-VIB   | vibracion   | mm/s   | TOR-01      | Linea 2   | activo             |   6.00 |  3.74 | 2026-10-30 10:13:46.586491+00 |       62.3 | normal
 TOR01-ENE   | energia     | kWh    | TOR-01      | Linea 2   | activo             |  25.00 | 17.06 | 2026-10-30 10:13:46.586804+00 |       68.2 | normal
 EMB02-TEMP  | temperatura | °C     | EMB-02      | Linea 3   | activo             |  75.00 | 61.44 | 2026-10-30 10:13:46.587151+00 |       81.9 | normal
 EMB02-VIB   | vibracion   | mm/s   | EMB-02      | Linea 3   | activo             |   5.00 |  2.80 | 2026-10-30 10:13:46.588478+00 |       56.0 | normal
 EMB02-ENE   | energia     | kWh    | EMB-02      | Linea 3   | activo             |  20.00 | 14.73 | 2026-10-30 10:13:46.58885+00  |       73.7 | normal
(9 rows)
```

Vistas creadas en PostgreSQL.

Evidencia 4 — P4: el usuario de Grafana no puede leer la tabla `lecturas`, pero sí la vista.

```
C:\Users\ASLHY\Downloads\iot-vistas\iot-vistas>docker exec vistas-datos psql -U grafana_lector -d iotanalytics -c "SELECT COUNT(*) FROM lecturas"
ERROR:  permission denied for table lecturas

C:\Users\ASLHY\Downloads\iot-vistas\iot-vistas>docker exec vistas-datos psql -U grafana_lector -d iotanalytics -c "SELECT COUNT(*) FROM v_estado_sensores"
 count
-------
     9
(1 row)
```

Seguridad: acceso solo a través de vistas.

Evidencia 5 — P5: el operador cambia TOR-01 a mantenimiento; la vista rechaza el estado inactivo por el CHECK OPTION y la tabla rechaza el acceso directo.

```
C:\Users\ASLHY\Downloads\iot-vistas\iot-vistas>docker exec vistas-datos psql -U operador_planta -d iotanalytics -c "UPDATE v_operacion_dispositivos SET esta
do='mantenimiento' WHERE codigo='TOR-01'"
UPDATE 1

C:\Users\ASLHY\Downloads\iot-vistas\iot-vistas>docker exec vistas-datos psql -U operador_planta -d iotanalytics -c "UPDATE v_operacion_dispositivos SET esta
do='inactivo' WHERE codigo='EMB-02'"
ERROR:  new row violates check option for view "v_operacion_dispositivos"
DETAIL:  Failing row contains (estado) = (inactivo).

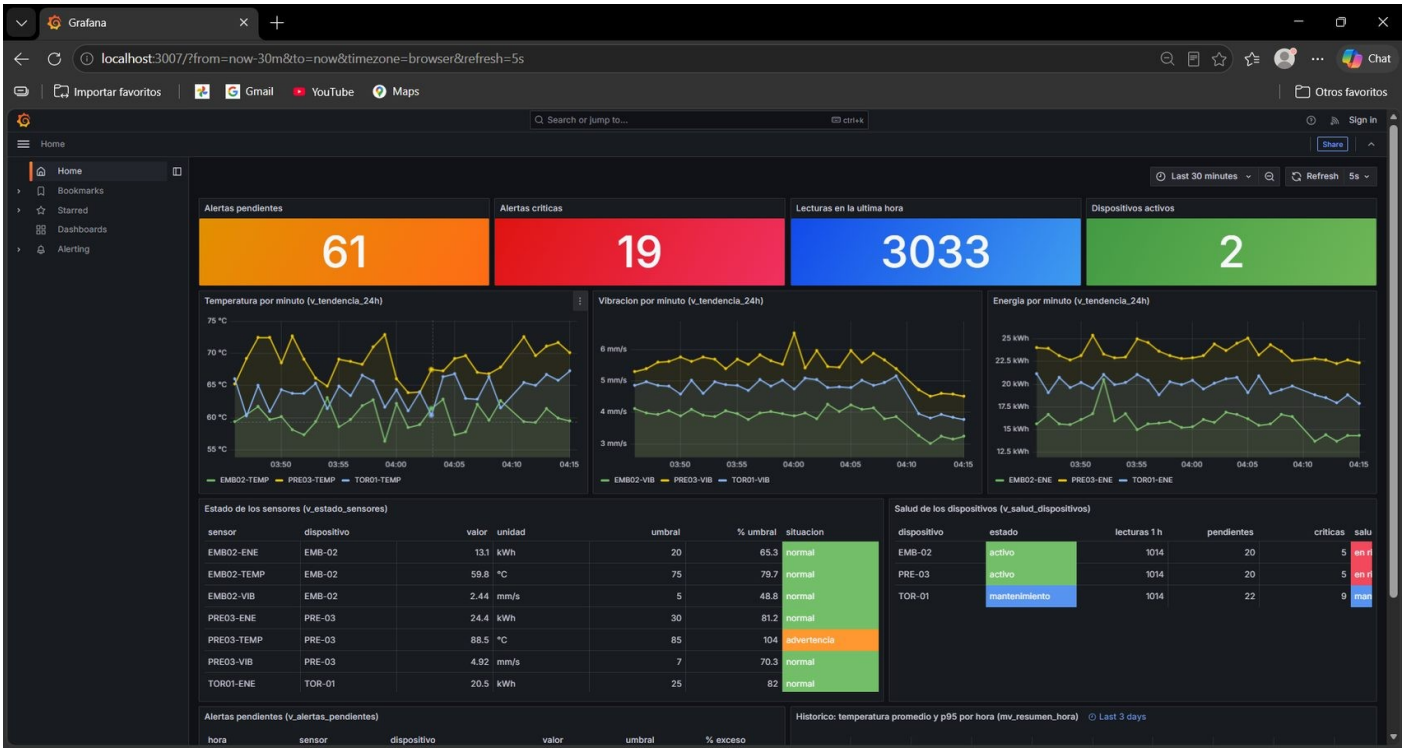
C:\Users\ASLHY\Downloads\iot-vistas\iot-vistas>docker exec vistas-datos psql -U operador_planta -d iotanalytics -c "UPDATE dispositivos SET estado='activo'"

ERROR:  permission denied for table dispositivos

C:\Users\ASLHY\Downloads\iot-vistas\iot-vistas>
```

Vista actualizable con WITH CHECK OPTION.

Evidencia 6 — P6: el cambio se refleja en el tablero: TOR-01 en mantenimiento y 2 dispositivos activos.



Tablero después del cambio de estado.

Evidencia 7 — P7: la misma información tarda 476.5 ms consultando la tabla y 2.4 ms desde la vista materializada.

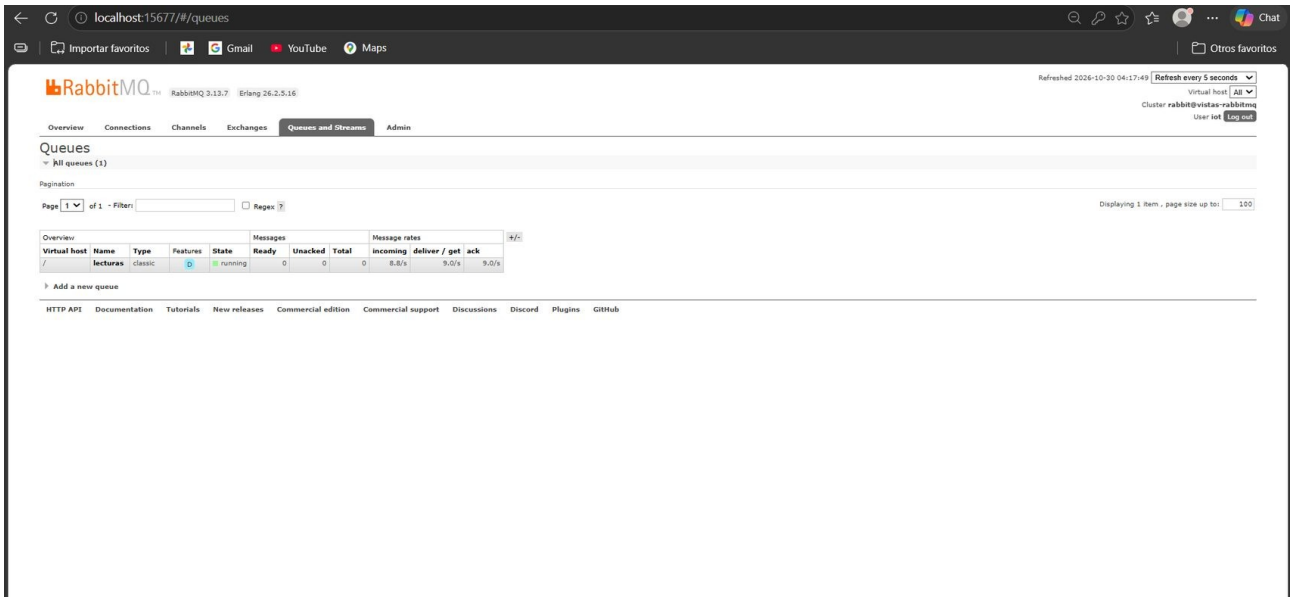
```
C:\Windows\System32\cmd.e
C:\Users\ASLHY\Downloads\iot-vistas\iot-vistas>docker exec -i vistas-datos psql -U postgres -d iotanalytics < datos/pruebas.sql
Timing is on.
--- Consulta directa sobre lecturas (3 dias, agrupado por hora) ---
                                  QUERY PLAN
------------------------------------------------------------------------------
 HashAggregate (actual time=474.317..474.769 rows=219 loops=1)
   Group Key: date_trunc('hour'::text, l.medido_en), s.tipo
   Batches: 1  Memory Usage: 913kB
   ->  Hash Join (actual time=0.115..260.360 rows=80190 loops=1)
         Hash Cond: (l.sensor_id = s.id)
         ->  Seq Scan on lecturas l (actual time=0.028..53.680 rows=80190 loops=1)
         ->  Hash (actual time=0.040..0.042 rows=9 loops=1)
               Buckets: 1024  Batches: 1  Memory Usage: 9kB
               ->  Seq Scan on sensores s (actual time=0.010..0.013 rows=9 loops=1)
 Planning Time: 6.768 ms
 Execution Time: 476.481 ms
(11 rows)

Time: 507.203 ms
--- Misma informacion desde mv_resumen_hora ---
                                QUERY PLAN
---------------------------------------------------------------------------
 HashAggregate (actual time=1.965..2.290 rows=219 loops=1)
   Group Key: hora, tipo
   Batches: 1  Memory Usage: 160kB
   ->  Seq Scan on mv_resumen_hora (actual time=0.016..0.237 rows=657 loops=1)
 Planning Time: 19.882 ms
 Execution Time: 2.416 ms
(6 rows)

Time: 32.423 ms
--- v_estado_sensores: el LATERAL usa el indice (1 fila por sensor) ---
                                        QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 Nested Loop Left Join (actual time=11.626..11.787 rows=9 loops=1)
   ->  Hash Join (actual time=0.320..0.339 rows=9 loops=1)
         Hash Cond: (s.dispositivo_id = d.id)
         ->  Seq Scan on sensores s (actual time=0.091..0.096 rows=9 loops=1)
         ->  Hash (actual time=0.034..0.035 rows=3 loops=1)
               Buckets: 1024  Batches: 1  Memory Usage: 9kB
               ->  Seq Scan on dispositivos d (actual time=0.016..0.019 rows=3 loops=1)
```

Comparación de rendimiento con EXPLAIN ANALYZE.

Evidencia 8 — P8: la cola `lecturas` de RabbitMQ recibe y entrega 9 mensajes por segundo, sin acumularse.



Cola de RabbitMQ en tiempo real.

El consumidor guarda las lecturas y refresca la vista materializada cada 30 segundos.

```
C:\Users\ASLHY\Downloads\iot-vistas\iot-vistas>
C:\Users\ASLHY\Downloads\iot-vistas\iot-vistas>docker compose logs --tail 5 consumidor
vistas-consumidor  | 2052 lecturas guardadas · mv_resumen_hora refrescada en 765 ms
vistas-consumidor  | 2322 lecturas guardadas · mv_resumen_hora refrescada en 1652 ms
vistas-consumidor  | 2601 lecturas guardadas · mv_resumen_hora refrescada en 595 ms
vistas-consumidor  | 2853 lecturas guardadas · mv_resumen_hora refrescada en 545 ms
vistas-consumidor  | 3123 lecturas guardadas · mv_resumen_hora refrescada en 360 ms

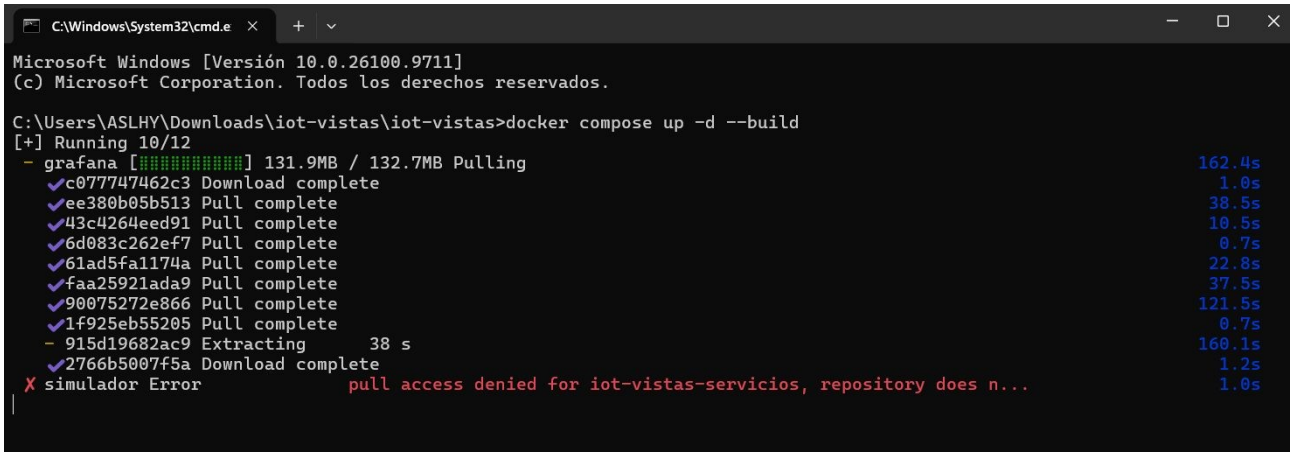
C:\Users\ASLHY\Downloads\iot-vistas\iot-vistas>
```

Registro del consumidor.

7. Problemas encontrados y solución

7.1 La imagen del simulador no existía

Al levantar el entorno por primera vez, Docker intentó descargar la imagen `iot-vistas-servicios` de Docker Hub y falló con “pull access denied”. El consumidor y el simulador usan la misma imagen, pero solo el consumidor tenía la instrucción `build`, y Docker Compose intentó descargar la del simulador antes de construirla.

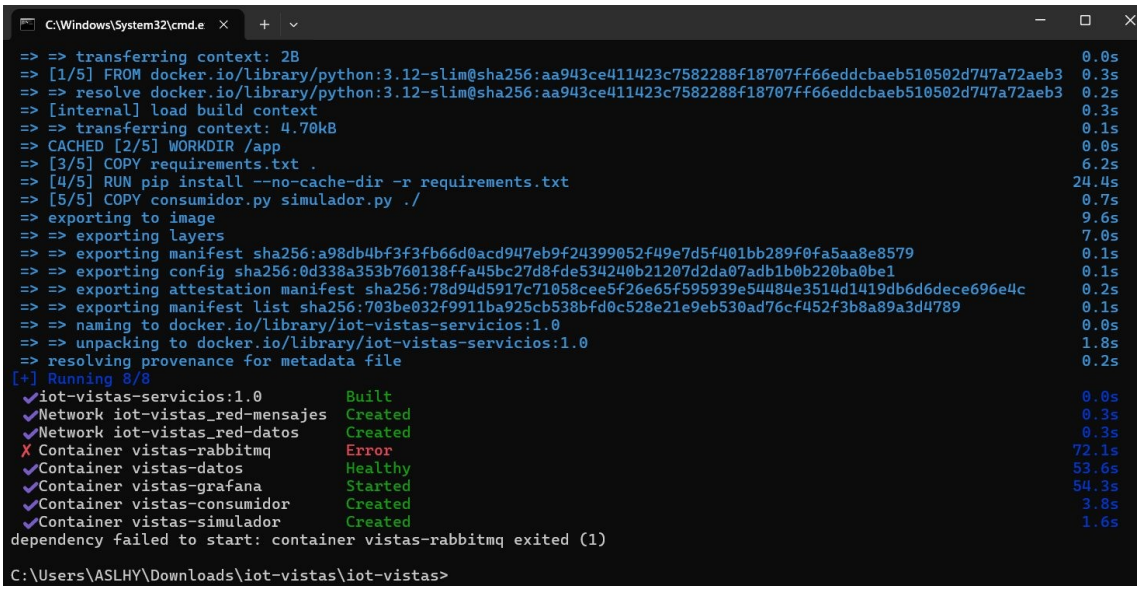


Error al descargar la imagen del simulador.

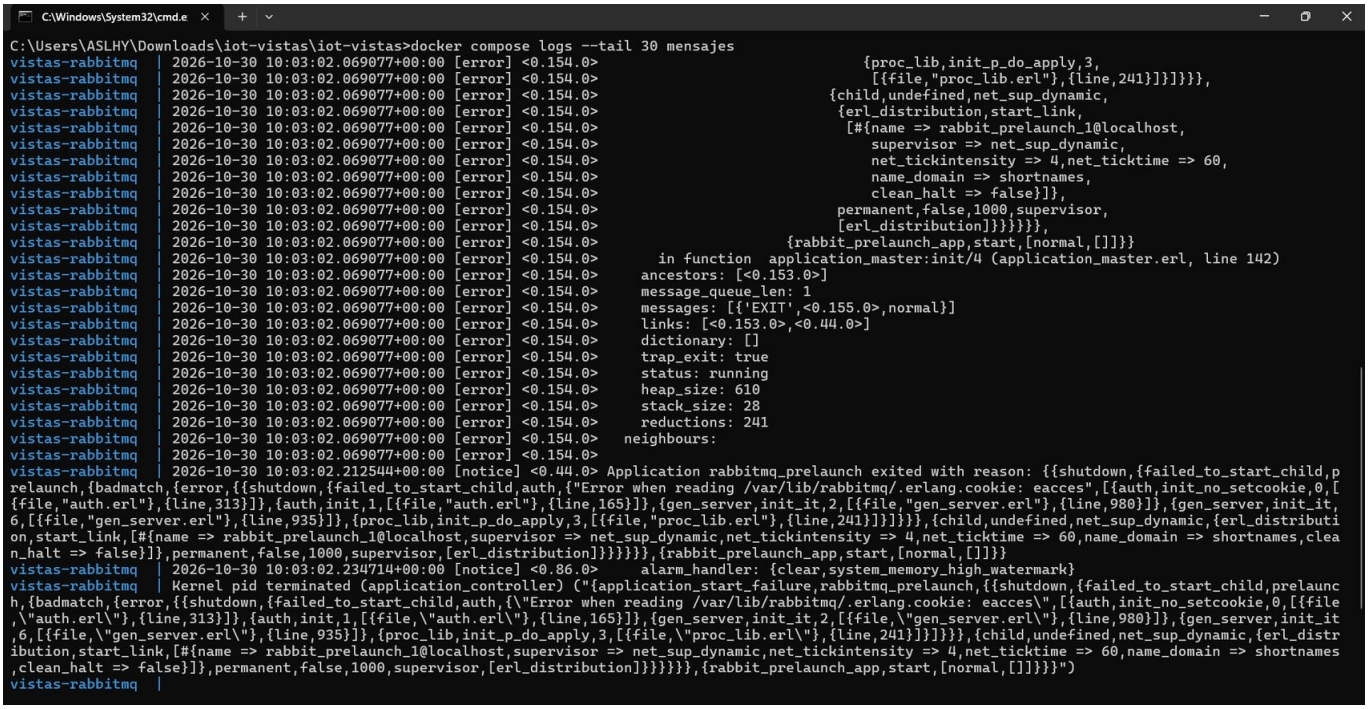
Solución: agregar `build: ./servicios` también al simulador, para que Compose sepa construir la imagen en lugar de buscarla en internet.

7.2 RabbitMQ no arrancaba (eacces en .erlang.cookie)

RabbitMQ terminaba con código 1 y el resto de los servicios no iniciaba. El registro mostraba “Error when reading /var/lib/rabbitmq/.erlang.cookie: eacces”: RabbitMQ no tenía permiso para leer su archivo interno de autenticación. Recrear solo el contenedor no lo resolvió.



RabbitMQ termina con error y detiene a los servicios que dependen de él.



Registro de RabbitMQ con el error eacces.

Era el mismo error que apareció en la Tarea 5, así que se aplicó la solución documentada ahí: pasar la cookie como argumento con `RABBITMQ_SERVER_ADDITIONAL_ERL_ARGS`, montar la carpeta de datos en memoria con permisos abiertos y cambiar el chequeo de salud por una prueba del puerto 5672, que no usa la cookie. Después del cambio RabbitMQ arrancó healthy (Evidencia 1).

```
    hostname: vistas-rabbitmq
    environment:
      RABBITMQ_SERVER_ADDITIONAL_ERL_ARGS: "-setcookie iotvistascookie"
    tmpfs:
      - /var/lib/rabbitmq:mode=1777
    healthcheck:
      test: ["CMD-SHELL", "bash -c '</dev/tcp/localhost/5672'"]
```

8. Control de versiones y colaboración

La actividad se desarrolló de forma individual. El proyecto se publicó en un repositorio público con un README que describe cada vista, los usuarios y cómo levantar el entorno: https://github.com/luisgael-ml/iot-vistas

```
git init
git config user.name "Luis Gael Monterrubio López"
git config user.email "monterrubiolopezluisgael@gmail.com"
git add .
git commit -m "Vistas en PostgreSQL para IoT Analytics Dashboard"
git branch -M main
git remote add origin https://github.com/luisgael-ml/iot-vistas.git
git push -u origin main
```

Evidencia 9: el commit se registró a nombre de Luis Gael Monterrubio López y se publicó en GitHub.

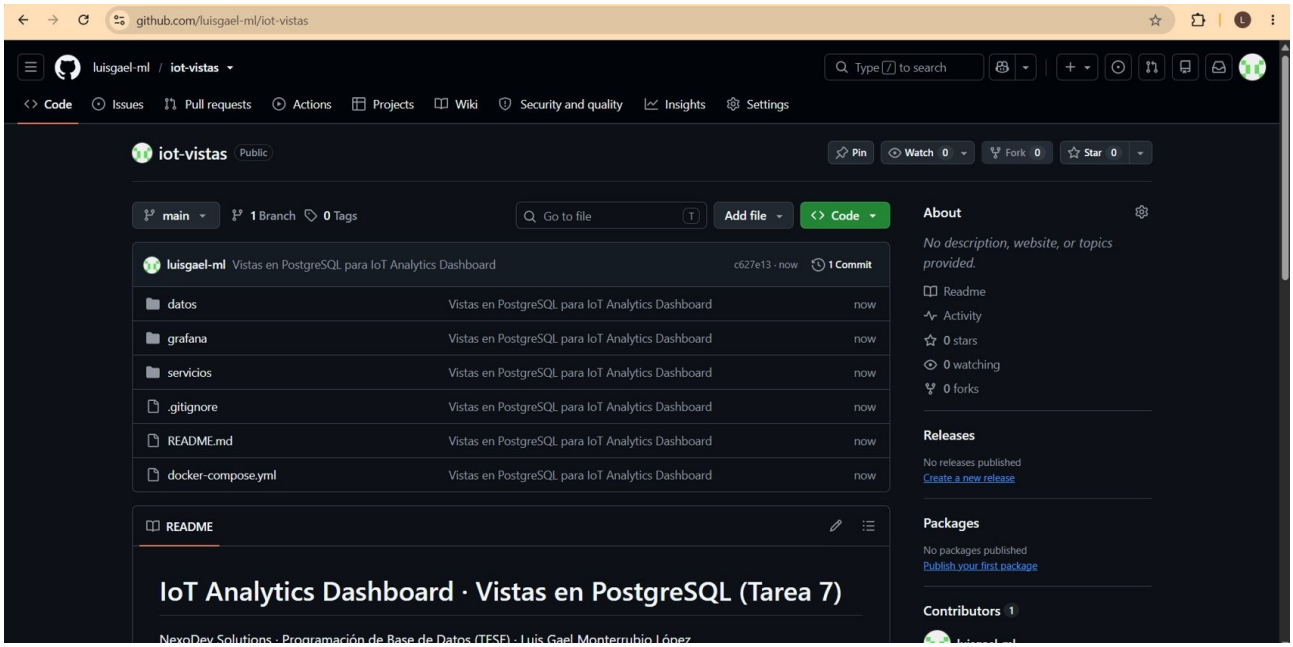
```
C:\Users\ASLHY\Downloads\iot-vistas\iot-vistas>git branch -M main

C:\Users\ASLHY\Downloads\iot-vistas\iot-vistas>git remote add origin https://github.com/luisgael-ml/iot-vistas.git

C:\Users\ASLHY\Downloads\iot-vistas\iot-vistas>git push -u origin main
Enumerating objects: 21, done.
Counting objects: 100% (21/21), done.
Delta compression using up to 4 threads
Compressing objects: 100% (16/16), done.
Writing objects: 100% (21/21), 11.13 KiB | 474.00 KiB/s, done.
Total 21 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/luisgael-ml/iot-vistas.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

C:\Users\ASLHY\Downloads\iot-vistas\iot-vistas>
```

Publicación del proyecto en GitHub.



Repositorio iot-vistas con la base de datos, los servicios y Grafana.

8.1 Reflexión individual sobre la colaboración

Las vistas funcionan como un acuerdo entre quien administra la base de datos y quien construye el tablero: el equipo de visualización trabaja con nombres y columnas estables, como `v_estado_sensores`, sin depender de cómo están organizadas las tablas. Si después cambio la estructura, solo ajusto la vista y el tablero sigue funcionando, lo que permite que cada parte del equipo avance sin bloquear a la otra.

Durante el proyecto aproveché la retroalimentación de trabajos anteriores: el error de RabbitMQ ya estaba documentado en la Tarea 5, y gracias a eso lo resolví rápido. Por eso en esta tarea también dejé documentados los dos problemas y su solución, para que un compañero que los encuentre no empiece de cero. El README, los comentarios en `init.sql` y la tabla de diseño explican por qué tomé cada decisión, de modo que cualquier integrante pueda revisarlas y proponer mejoras, por ejemplo cada cuánto refrescar la vista materializada.

9. Conclusiones

Las vistas permitieron separar lo que el tablero necesita ver de cómo están guardados los datos. Grafana pasó de consultas con uniones, agrupaciones y cálculos a un simple `SELECT` sobre una vista, y las reglas como la severidad de una alerta o la salud de un dispositivo quedaron definidas en un solo lugar.

En seguridad, las vistas junto con los permisos aplicaron el principio de mínimo privilegio: Grafana solo lee vistas, el operador solo cambia estados permitidos gracias a `WITH CHECK OPTION`, y el consumidor solo inserta lecturas. Las pruebas confirmaron que cualquier intento de acceso directo a las tablas es rechazado.

En rendimiento, una vista normal no acelera una consulta por sí sola, pero un buen diseño sí: el `LATERAL` con índice obtiene la última lectura sin recorrer la tabla, y la vista materializada redujo una consulta histórica de 476 ms a 2.4 ms. La integración con RabbitMQ y Grafana mostró que las vistas funcionan con datos que cambian cada segundo, cumpliendo el objetivo de NexoDev de un acceso a datos IoT más simple, seguro y eficiente.

Referencias

Docker Inc. (s.f.). *Networking in Compose*. Docker Docs. https://docs.docker.com/compose/how-tos/networking/

Grafana Labs. (s.f.). *PostgreSQL data source*. Grafana documentation. https://grafana.com/docs/grafana/latest/datasources/postgres/

PostgreSQL Global Development Group. (s.f.). *CREATE MATERIALIZED VIEW*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/sql-creatematerializedview.html

PostgreSQL Global Development Group. (s.f.). *CREATE VIEW*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/sql-createview.html

PostgreSQL Global Development Group. (s.f.). *GRANT*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/sql-grant.html

PostgreSQL Global Development Group. (s.f.). *REFRESH MATERIALIZED VIEW*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/sql-refreshmaterializedview.html

PostgreSQL Global Development Group. (s.f.). *Rules and privileges*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/rules-privileges.html

RabbitMQ. (s.f.). *Clustering guide: Erlang cookie*. RabbitMQ documentation. https://www.rabbitmq.com/docs/clustering